Impala acceleration: results and lessons

Ayan Biswas ayanju04@gmail.com

September 30, 2026 — Steps 1–9

What we achieved

For the tested pooled copper calibration, the final experimental GPU workflow completed four chains in **154.17 seconds**, compared with **205.90 seconds on the optimized parallel CPU** and a historical **962.03-second original sequential reference**.

Implementation	Sampling time	Relative throughput
Original CPU, four sequential chains	962.03 s	1.00×
Optimized CPU, four parallel chains	205.90 s	4.67×
Optimized GPU, four coordinated chains	154.17 s	6.24×

The fair incremental GPU comparison is 1.34× the optimized CPU throughput, or about 25% less elapsed time. The 6.24× historical gain combines CPU optimizations, parallel chains, GPU execution and worker improvements; it must not be described as a GPU-only speedup.

Benchmark and comparison limits

The long benchmark uses four independent seeds (42, 123, 456, 789), 40,000 stored states per chain and 50 temperatures, with the existing pooled Cu/PTW example. Each chain has its own random-number stream and adaptation state. Only forward-model requests are combined for GPU evaluation.

The hardware is a Core Ultra 9 285HX CPU and RTX PRO 5000 Blackwell laptop GPU. One numerical-library thread is used per worker. Timings exclude startup/JIT, saving and diagnostics, while including initialization inside sampling, cache misses, CPU work, transfers and coordination. The original reference is a historical sum of sequential chain times; current CPU/GPU timings are group wall times.

The final CPU/GPU figures come from the same step-8 comparison. Each long backend was measured once; repeated short controls provide supporting evidence. Results are specific to this workload and machine. They are not universal performance claims or uncertainty bounds.

Practical status

The CPU implementation remains available. GPU support is an **optional, experimental benchmark path for the tested Cu model**, not a finished general-purpose production backend. GPU adapter cleanup, automatic backend selection/failover and Mac validation remain future work. Sampler distributions, proposal order and adaptation rules were preserved. Hierarchical mixing was not changed.

How the improvements were achieved

Change	Why it helped and what the measurements showed
Cache fixed preparation	Reused experimental geometry and history inputs instead of rebuilding them. Modest early gains; original fixture outputs stayed exact.
Compile CPU history loop	Used an optional Numba driver for the repeated material history loop. Pooled short-run sampling was about 1.97–1.98× faster than the cached Python path in the step-2 tests.
Batch independent chains	Combined separate chains' forward-model requests into larger GPU calls, preserving each chain's RNG and sampler state. Batching alone was insufficient: the first version still lost to parallel CPU.
Keep geometry on the GPU	Cached device geometry, performed interpolation on the GPU and returned only needed predictions. This was the first coordinated GPU version to beat the parallel CPU in the long benchmark.
Tune history kernel	Selected 32-thread blocks and moved invariant calculations outside the integration loop, preserving float64 arithmetic.
Batch Gibbs bookkeeping	Vectorized repeated temperature-wise noise and likelihood calculations on CPU while retaining draw order and RNG state. This reduced the CPU work feeding the GPU.
Batch proposal likelihoods	Reduced scalar likelihood calls, but delivered only a small additional GPU gain. CPU timing evidence was mixed; kept opt-in.

Measured milestones: fresh controls within each step

Step	Change	Previous GPU	New GPU
5	Resident geometry/interpolation	–	214.32 s
6	Kernel tuning	213.56 s	202.71 s
7	Batched Gibbs bookkeeping	196.58 s	157.59 s
8	Batched proposal likelihoods	156.06 s	154.17 s

Step 5 beat its fresh parallel CPU control (252.16 s) by 17.7% in throughput. Step 6 improved GPU throughput by 5.4%; step 7 by 24.7%; step 8 by 1.2%. The step-8 short median gain was 3.0%, and CPU results were inconsistent. **Do not multiply these percentages**: each step remeasured its controls, and operating conditions/timing variation changed between runs.

What did not work well

The first history-only GPU implementation was slower end-to-end than compiled CPU for a single chain (211.88 versus 186.68 s). Initial four-chain GPU batching was also slower than parallel CPU (334.53 versus 252.64 s). Reducing host work and transfers mattered more than merely moving the history calculation to a GPU. The final small proposal-batching gain does not establish a robust general speedup.

How stochastic accuracy was checked

We measured **effective samples per second (ESS/s)**, not just iterations per second. Faster iterations are useful only if posterior quality and sample correlation remain acceptable.

Validation combined an unchanged 256-case original prediction/likelihood fixture, seeded comparisons against frozen original CPU traces, full-temperature short-run checks and four-chain long-run posterior diagnostics. The long diagnostics discard 20,000 states per chain, without thinning, and cover 16 parameter/noise quantities.

Diagnostic	Required	Final GPU result
Maximum R-hat	< 1.01	1.002585
Minimum bulk ESS	≥ 400	2253.48
Minimum tail ESS	≥ 400	3345.25
Minimum bulk ESS/s	Comparative metric	14.62

The optimized CPU delivered 10.94 minimum bulk ESS/s in the same step-8 study, so the GPU improvement was about 34%. MCSE/SD, posterior-mean differences and prediction-mean differences also passed the predeclared gates.

Optimized CPU parameter/noise/counter/prediction arrays reproduced the frozen original CPU traces exactly. GPU parameter draws and counters matched CPU exactly; noise and prediction differences were at floating-point rounding scale and passed unchanged tolerances. Later GPU changes preserved saved GPU outputs bit-for-bit, so their measured throughput gains also improved ESS/s for those runs.

The latest complete hardware-enabled regression suite passed **135 tests**. Step 9 then passed 19 focused instrumentation/GPU tests, and all saved arrays matched across 12 successful profiling runs. These are separate test runs, not 154 distinct tests. Bitwise agreement is evidence for these tested environments and seeds, not a promise across hardware or numerical-library versions.

Why we stopped GPU optimization here

Step 9 separated overlapping timers. In a representative standard instrumented run, GPU kernels accounted for 49.8% of elapsed time, other model work 17.8%, waiting for worker requests 24.3%, and request decoding/reply sending 6.6%. Assembly and remaining bookkeeping accounted for about 1.6%. Polling includes necessary sampler work and cannot simply be removed. Detailed instrumentation itself increased median runtime by about 2.2%.

Larger batches revealed geometry preparation costs, mostly first-use setup rather than cache evictions. These findings identify future targets, but do not establish an asynchronous scheduling speedup. Given diminishing incremental returns, it is reasonable to move to other acceleration work. **Hierarchical mixing remains unresolved** and should be studied separately because it may require algorithm changes.

Where to find the evidence

Checkpoint: `perf-step9-20260930` (`33e7da6`); branch `perf/gpu-coordination-profile`. See the performance change log, `STEP1.md`–`STEP9.md` and `results/step*-results.json` for commands, measurements, diagnostic CSVs and SHA-256 manifests. Raw artifacts reside in workspace `benchmark/changes/` and `benchmark/results/reference/`. This consolidated PDF and its LaTeX source are included in the repository.